

Release Notes — v1.0.1

Frontend developer · 2026-07-04 · Material Master updates & new Project Manager role

Release date: 2026-07-04 **Backend version:** 1.0.1 **Audience:** Frontend developer

This release covers **Material Master** API changes (four) **plus a new Project Manager role** (§5). This doc is the frontend action list: what the API now sends/expects, and what to change in the UI.

These changes are live once the **backend v1.0.1 is deployed**. Coordinate the UI merge with that deploy — an old UI against v1.0.1 will send the removed `supplier_id` / `description` fields and get **400** or silently drop data.

At a glance

#	Change	What the frontend does
1	Material supports multiple preferred vendors	Vendor picker → multi-select, send <code>vendor_ids[]</code>
2	Material <code>description</code> → <code>remarks</code>	Rename the field on the material form + views
3	Material supports multiple resource specs (new master)	Add a resource-spec multi-select, send <code>resource_spec_ids[]</code>
4	Purchase-price history + auto-updating <code>last_purchase_price</code> / <code>date</code>	Show <code>last_purchase_date</code> ; add a price-history view
5	New Project Manager role (scoped projects, BOM release, line dates, stage)	Add PM login/nav + the new project/BOM-line endpoints

Affected endpoints: `POST /api/parts` , `PATCH /api/parts/:id` , `GET /api/parts/:id` , `GET /api/parts` , the new `GET /api/resource-specs` master (dropdown source), and the new `GET /api/parts/:id/price-history` . Goods receipt (`POST /api/purchase-orders/:id/receive`) now also updates the material price. §5 adds `PATCH /api/projects/:id/stage` and `PATCH /api/bom-lines/:id/required-date` , plus a `project_manager_id` on projects and a `required_by_date` on BOM lines.

1. Multiple preferred vendors per material

A material used to carry a single preferred vendor (`supplier_id`). It now carries a **list** of preferred vendors, `vendor_ids` .

Request — create / update

Send an **array of vendor UUIDs** instead of a single id:

```
// POST / PATCH /api/parts
{
  "name": "SS304 Ball Valve 15mm",
  "category_id": "...", "subtype_id": "...",
  "vendor_ids": ["<vendorUuid>", "<vendorUuid>"] // was: "supplier_id": "<uuid>"
}
```

Rules:

- `vendor_ids` is **optional**. Every id must be an existing vendor, else `400` .
- On `PATCH` : **omit** `vendor_ids` to leave the vendor list unchanged; send `[]` to clear all vendors.

Response — reads

`GET /api/parts/:id` and the create/update responses now include:

Field	Type	Use in UI
<code>vendor_ids</code>	<code>string[]</code>	Selected vendor UUIDs, in the order they were set — bind the multi-select to this
<code>preferred_vendors</code>	<code>Vendor[]</code>	Full vendor objects (name, code, ...) — render chips/list without a second lookup

`supplier_id` no longer exists on a material. Remove every read/write of it. (`GET /api/parts` list rows do **not** include the vendor arrays — fetch the single material for its vendors.)

UI work

- Change the material form's vendor control from **single-select** → **multi-select**.
- Submit the selection as `vendor_ids: string[]` .
- On the material detail view, render vendors from `preferred_vendors` (display) or `vendor_ids` (ids).

Type change (frontend **Part**)

```
- supplier_id: string | null
+ vendor_ids: string[]
+ preferred_vendors: Vendor[]
```

2. Material **description** renamed to **remarks**

The material's free-text field **description** is renamed to **remarks** (same free text, same 2000-char limit). Only the **name** changed.

Request & response

```
// POST / PATCH /api/parts and every material read
{
  "name": "SS304 Ball Valve 15mm",
  "remarks": "2-way ball valve" // was: "description"
}
```

- The material **search** query param still matches this field (now **remarks**).
- **Scope:** the **material** field only. **description** on **BOM lines** and **projects** is unchanged — do **not** rename those.

UI work

- Rename the material form field, label binding, and detail view from **description** → **remarks** .

Type change (frontend **Part**)

```
- description: string
+ remarks: string
```

3. Multiple resource specs per material (new)

Materials can now be tagged with one or more **resource specs** drawn from a new predefined master. This works exactly like preferred vendors — a multi-select backed by a master list.

New master endpoint — dropdown source

```
GET /api/resource-specs → [{ id, code, name, description, is_active }]
```

Admin-managed CRUD (create/update/delete are Administrator-only), same shape as Units/Grades. Use `GET` to populate the resource-spec multi-select.

Delete guard: `DELETE /api/resource-specs/:id` returns **409** if the spec is still assigned to any material — unassign it from those materials first. Handle this error in the admin UI.

Request — create / update

```
// POST / PATCH /api/parts
{
  "name": "SS304 Ball Valve 15mm",
  "resource_spec_ids": ["<resourceSpecUuid>", "<resourceSpecUuid>"]
}
```

Rules (identical to `vendor_ids`):

- Optional. Every id must be an existing resource spec, else **400**.
- On `PATCH`: **omit** to leave unchanged; send `[]` to clear.

Response — reads

`GET /api/parts/:id` and create/update responses include:

Field	Type	Use in UI
<code>resource_spec_ids</code>	<code>string[]</code>	Selected ids — bind the multi-select to this
<code>resource_specs</code>	<code>ResourceSpec[]</code>	Full objects (code, name, description) — render chips/list

List rows (`GET /api/parts`) do **not** include these — fetch the single material for its resource specs.

UI work

- Add a **resource-spec multi-select** to the material form, sourced from `GET /api/resource-specs`, submitting `resource_spec_ids: string[]`.
- Render them from `resource_specs` on the material detail view.

Type change (frontend **Part**)

```
+ resource_spec_ids: string[]  
+ resource_specs: ResourceSpec[]
```

4. Purchase-price history (new)

`last_purchase_price` is no longer just a static number the user types — it is now tracked over time and **auto-updates when the material is purchased**.

How it behaves

- **At creation:** the `last_purchase_price` you enter is captured as the opening value — `last_purchase_date` is stamped and an **Initial** row is added to the price ledger.
- **On goods receipt** (`POST /api/purchase-orders/:id/receive`): each received line's unit price becomes the material's `last_purchase_price`, `last_purchase_date` becomes the receipt date, `last_purchase_vendor_id` becomes the PO's vendor, and a **Purchase** row (vendor + PO + qty) is appended to the ledger.
- So `last_purchase_price` is **effectively read-only** — you can still send it on `PATCH`, but the next goods receipt overwrites it.

New fields on reads

`GET /api/parts/:id` now returns `last_purchase_date` (ISO timestamp, or `null`), `last_purchase_vendor_id`, and a resolved `last_purchase_vendor` object (the vendor of the last purchase, `null` for a manual opening value) — next to `last_purchase_price`.

New endpoint — price history

```
GET /api/parts/:id/price-history  
→ { part_id, last_purchase_price, last_purchase_date, last_purchase_vendor,  
    history: [ ... ] }
```

Each `history` row (newest first):

Field	Type	Notes
<code>unit_price</code>	string	Price at that event
<code>source</code>	"Initial" "Purchase"	Manual opening value vs. a vendor receipt
<code>vendor_id</code>	string null	Supplier (null for <code>Initial</code>)
<code>purchase_order_id</code>	string null	Source PO (null for <code>Initial</code>)
<code>reference</code>	string	PO number, or "Initial"
<code>quantity</code>	string	Accepted qty received
<code>effective_date</code>	string	When the price took effect

UI work

- Show `last_purchase_date` and `last_purchase_vendor` next to `last_purchase_price` on the material view ("last bought at X from Y on Z").
- Add a **price-history** panel/table from `GET /api/parts/:id/price-history` (e.g. a trend list or mini chart).
- Treat `last_purchase_price` as system-maintained after purchases (label it "last purchase" rather than an editable price).

Type change (frontend `Part`)

```
+ last_purchase_date: string | null
+ last_purchase_vendor_id: string | null
+ last_purchase_vendor: Vendor | null // resolved, on single-material reads
```

5. New role: Project Manager

A new user role `Project Manager` joins the existing set (Administrator, Engineering, Commercial, Purchase, Stores, Management). It is **scoped to the projects it is assigned to** and owns project coordination.

Assignment

Projects gain a `project_manager_id` field. Set it on create/update (`POST` / `PATCH /api/projects`) to assign the PM. `GET /api/projects/:id` now also returns `manager_name` / `manager_initials` /

`manager_hue` .

What a Project Manager can do

Capability	Endpoint
See only their projects & BOMs	<code>GET /api/projects</code> , <code>GET /api/project-boms</code> auto-filtered; foreign <code>:id</code> → 404
Release a BOM for purchase	<code>POST /api/project-boms/:id/transition</code> <code>{ "action":"advance" }</code> from the <code>Approved</code> stage (also allowed: Purchase)
Set each line's required-by date	<code>PATCH /api/bom-lines/:id/required-date</code> <code>{ "required_by_date":"YYYY-MM-DD" }</code> — works at any BOM stage
Update the project stage	<code>PATCH /api/projects/:id/stage</code> <code>{ "stage": <ProjectStage> }</code>

A PM may only act on projects/BOMs they're assigned to (else **403**). Material master and BOM-explorer reads stay open to all authenticated users.

New fields

- `Project` : `project_manager_id` (+ `manager_*` on detail reads).
- `BomLine` : `required_by_date` (`YYYY-MM-DD` or `null`).

UI work

- Add **Project Manager** to the role picker (user management) and the sidebar/route guard map (`navForRole`).
- Add a PM assignment control on the project form (`project_manager_id`).
- For a logged-in PM, the project/BOM lists are already scoped — no client filtering needed; just render what returns.
- Add a **release-for-purchase** action (BOM at `Approved`) and a **project stage** control for the PM.
- Add a **required-by date** field per BOM line (PM-editable at any stage).

Type changes

```
// Role
+ "Project Manager"
// Project
+ project_manager_id: string | null
+ manager_name / manager_initials / manager_hue // detail reads
// BomLine
+ required_by_date: string | null // YYYY-MM-DD
```

Migration checklist (frontend)

- ☐ Material form: vendor picker → **multi-select**, submit `vendor_ids: string[]`.
- ☐ Material detail: render vendors from `preferred_vendors`.
- ☐ Remove all uses of `supplier_id` on materials.
- ☐ Material form + views: rename `description` → `remarks`.
- ☐ Material form: add a **resource-spec multi-select** from `GET /api/resource-specs`, submit `resource_spec_ids: string[]`.
- ☐ Material detail: render resource specs from `resource_specs`.
- ☐ Update the `Part` type/interface (`vendor_ids` , `preferred_vendors` , `remarks` , `resource_spec_ids` , `resource_specs` , `last_purchase_date`).
- ☐ Show `last_purchase_date` and add a price-history view from `GET /api/parts/:id/price-history` ; treat `last_purchase_price` as system-maintained.
- ☐ Add **Project Manager** to the role picker + `navForRole` ; add project PM assignment (`project_manager_id`).
- ☐ Wire PM actions: release-for-purchase (BOM `Approved`), project stage (`PATCH /projects/:id/stage`), and per-line required-by date (`PATCH /bom-lines/:id/required-date`).
- ☐ Ship the UI **together with** the backend v1.0.1 deploy.

Quick reference — material field mapping

v1.0.0 (old)	v1.0.1 (new)	Notes
supplier_id: string	vendor_ids: string[]	Single → list of vendor UUIDs
—	preferred_vendors: Vendor[]	New; single-material reads only
description: string	remarks: string	Rename only
—	resource_spec_ids: string[]	New; multi-select from /api/resource-specs
—	resource_specs: ResourceSpec[]	New; single-material reads only
—	last_purchase_date: string null	New; auto-set on purchase
—	last_purchase_vendor_id + last_purchase_vendor	New; vendor of the last purchase
last_purchase_price (manual)	last_purchase_price (auto)	Now auto-updates on goods receipt

Deeper rationale and the full v1.0.1 change list (including non-material backend changes) live in [BACKEND_CHANGES_FOR_FRONTEND.md](#) ; the complete endpoint contract is in [FRONTEND_API_GUIDE.md](#) .